

Class Test – 1
CSE 3105 - Microprocessors and Embedded System

1. What is the difference between a microprocessor and a microcontroller?

2

Ans: A **microprocessor** is basically the CPU on a single chip. It contains only the processing unit and requires external components like memory (RAM/ROM), input-output ports, and timers to form a complete system. Because of this, microprocessors are mainly used in general-purpose computing systems such as personal computers and laptops, where flexibility and processing power are more important than compact design.

On the other hand, a **microcontroller** is a complete system on a chip. It integrates the CPU along with memory, input-output ports, and peripheral devices within a single IC. This makes microcontrollers compact, low-cost, and power-efficient, which is why they are widely used in embedded systems like washing machines, microwave ovens, automobiles, and IoT devices.

In short, the microprocessor is like the “brain only,” while the microcontroller is like a “mini computer on a chip” designed for dedicated control tasks.

2. Describe the function of the Bus Interface Unit (BIU) and Execution Unit (EU).

3

Ans: The **Bus Interface Unit (BIU)** is responsible for interacting with memory and input/output devices. It fetches instructions, reads or writes data from/to memory, and manages the instruction queue. In the 8086 microprocessor, the BIU works in the background to prefetch instructions and store them in a queue, which increases the speed of execution by overlapping instruction fetch and execution.

The **Execution Unit (EU)**, on the other hand, is responsible for decoding and executing the instructions fetched by the BIU. It performs arithmetic, logical, and control operations using the ALU, general-purpose registers, and flag register. The EU cannot directly access memory or I/O—it depends on the BIU to provide the required instructions and data.

3. Explain how physical addresses are calculated in 8086 with an example.

3

Ans: In the microprocessor, physical addresses are calculated using segment and offset values. Each time the processor wants to access memory, it takes the contents of a segment register, shifts it one hexadecimal place to the left (same as multiplying by 16_{10}), then add the required offset to form the 20-bit address. A physical address is obtained by the formula.

$$\text{Physical Address} = (\text{Segment} \times 16) + \text{Offset}$$

- **Segment:** 16-bit value that points to the start of the segment.
- **Offset:** 16-bit value that specifies the distance from the segment start.

Example:

If **CS = 2000h** (code segment) and **IP = 1234h** (instruction pointer), the physical address of the next instruction is:

$$\text{Physical Address} = (2000h \times 16) + 1234h = 20000h + 1234h = 21234h$$

This physical address points to the actual location in memory where the instruction or data is stored.

Why we do it?

We shift the segment register one hexadecimal place to the left (multiply by 16) to convert the 16-bit segment value into a 20-bit base address in memory.

Here's why:

- Memory in 8086 is 1 MB (2^{20} bytes), but segment registers are 16 bits.
- If we just used the 16-bit value directly, it could only address 64 KB (2^{16} bytes).
- By shifting left 4 bits ($\times 16$), we create overlapping segments and can combine it with a 16-bit offset to form a full 20-bit physical address.

4. List and explain the general-purpose registers of 8086 microprocessor.

2

The 8086 microprocessor has **eight general-purpose registers**, each **16 bits wide**, which can also be accessed as two separate 8-bit registers (high and low bytes). These registers are mainly used for arithmetic, logic, and data manipulation operations.

- **AX register** – It is also known as accumulator register. It is used to store operands for arithmetic operations.
- **BX register** – It is used as a base register. It is used to store the starting base address of the memory area within the data segment.
- **CX register** – It is referred to as counter. It is used in loop instruction to store the loop counter.
- **DX register** – This register is used to hold I/O port address for I/O instruction.
- **Stack pointer register** -It is a 16-bit register that holds the offset address of the top of the stack in the stack segment, and it is automatically updated during PUSH and POP operations.
- **Base Pointer(BP)** - It is used to store the base address of the memory.Used to access parameters or data on the Stack Segment (SS)
- **Source Index(SI)** - It is a memory pointer which is used to store the offset address of the source.
- **Destination Index(DI)** - It is a memory pointer which is used to store the offset address of the destination.

Class Test – 2

CSE 3105 - Microprocessors and Embedded System

1. Briefly explain the general-purpose registers of 8086 microprocessor.

5

Or

Define the term *segment register* and explain four segment registers in 8086 microprocessor.

Ans: A segment register in the 8086 microprocessor is a 16-bit register that holds the starting address (base) of a memory segment. Since the 8086 uses segmented memory, each segment

can be up to 64 KB, and the segment register helps calculate the physical address when combined with an offset. There are four main segment registers in 8086:

1. **CS (Code Segment):** The Code Segment (CS) register is a 16-bit segment register in the 8086 that holds the starting address of the code segment in memory, and it is always used with the Instruction Pointer (IP) to access the next instruction to be executed
2. **DS (Data Segment):** The Data Segment (DS) is a 16-bit register that points to the current data segment, where program data is stored, and operands for most instructions are fetched from this segment
3. **SS (Stack Segment):** The Stack Segment (SS) register is a 16-bit register that points to the current stack segment in memory, and it works with the Stack Pointer (SP) or Base Pointer (BP) to access data in the stack
4. **ES (Extra Segment):** The Extra Segment (ES) register is a 16-bit segment register that holds the base address of the extra data segment, and it is mainly used with the Source Index (SI) or Destination Index (DI) in string and memory operations. ES allows access to additional data beyond that 64KB

2. What are the conditional flags and carry flags of 8086 microprocessor?

5

Ans: In the 8086 microprocessor, **flags** are special bits in the **Flag Register** that indicate the status of the processor after arithmetic or logical operations. Among them, **conditional flags** and the **carry flag** are important for decision-making.

1. **Conditional Flags:** These flags reflect the result of an operation and are used for conditional jumps or decisions:
 - **ZF (Zero Flag):** Set to 1 if the result of an operation is zero.
 - **SF (Sign Flag):** Set to 1 if the result is negative (most significant bit = 1).
 - **PF (Parity Flag):** Set to 1 if the number of 1s in the result is even.
 - **OF (Overflow Flag):** Set to 1 if the result of a signed operation exceeds the allowed range.
2. **Carry Flag (CF):** This flag is set when an arithmetic operation generates a **carry out** from the most significant bit (for addition) or a **borrow** (for subtraction). It is mainly used in **multi-byte arithmetic** and in **instructions like ADC, SBB, and rotate/shift operations**.

Or

Explain how physical addresses are calculated in 8086 with an example.

Or

Evaluate how the use of segmented memory benefits the 8086 microprocessors.

Ans: The use of segmented memory in the 8086 microprocessor provides several important benefits. **First**, it allows the processor to access more than 64 KB of memory using only 16-bit registers, because the combination of a segment address and an offset generates a 20-bit physical address, enabling access to 1 MB of memory. **Second**, it supports modular programming, as code, data, and stack can be

placed in separate segments, making programs easier to manage and organize. **Third**, segmented memory allows overlapping segments, giving flexibility in memory allocation and efficient use of available memory. **Finally**, it improves execution speed, because the processor can prefetch instructions into a queue from one segment while executing instructions from another.

3. Write the syntax of the AND, TEST, and XCHG instructions in 8086. 5

Ans: **Syntax of AND, TEST, and XCHG Instructions in 8086 (with examples)**

1. AND Instruction

- **Syntax:** AND destination, source
- **Function:** Performs bitwise AND of source and destination, stores result in destination.
- **Example:**

MOV AL, 5 ; AL = 00000101b

AND AL, 3 ; AL = 00000101b AND 00000011b = 00000001b

Result: AL = 1

2. TEST Instruction

- **Syntax:** TEST destination, source
- **Function:** Performs bitwise AND, **does not store the result**, only updates flags (ZF, SF, PF).
- **Example:**

MOV AL, 5 ; AL = 00000101b

TEST AL, 1 ; AND AL with 00000001b, result not stored

Result: Zero Flag (ZF) = 0, since result $\neq$ 0

3. XCHG Instruction

- **Syntax:** XCHG destination, source
- **Function:** Exchanges (swaps) contents of destination and source.
- **Example:**

MOV AL, 5 ; AL = 5

MOV BL, 10 ; BL = 10

XCHG AL, BL ; AL $\leftrightarrow$ BL

Result: AL = 10, BL = 5

Or

Differentiate between ADD and ADC instructions.

Difference Between ADD and ADC Instructions

1. ADD Instruction

- **Description:** Adds the **source** to the **destination**. Carry flag is updated based on the result. Used for simple addition.

- **Example (your style):**

ADD AL, data8 ; AL = AL + data8

ADD AX, data16 ; AX = AX + data16

(reg2) → (reg2) + (reg1)

2. ADC Instruction

- **Description:** Adds the **source**, **destination**, and **Carry flag (CF)** together. Used for multi-byte or multi-word addition to include previous carry.

- **Example (your style):**

ADC reg2, reg1 ; reg2 = reg2 + reg1 + CF

ADC reg2, mem ; reg2 = reg2 + mem + CF

ADC mem, reg1 ; mem = mem + reg1 + CF

ADC A, data ; Add to accumulator with carry

(reg2) → (reg2) + (reg1) + CF

4. Explain how the based indexed addressing mode works with an example.

5

In Based Addressing, **BX** or **BP** is used to hold the base value for the effective address, and a signed 8-bit or unsigned 16-bit displacement is specified in the instruction.

In case of an 8-bit displacement, it is sign-extended to 16-bit before adding to the base value.

When **BX** holds the base value of EA, the 20-bit physical address is calculated using **BX** and **DS**.

When **BP** holds the base value of EA, the 20-bit physical address is calculated using **BP** and **SS**.

Example:

MOV AX, [BX + 08H]

Operations:

0008H ← 08H (Sign extended)

EA = (BX) + 0008H

BA = (DS) × 10H

MA = BA + EA

$(AX) \leftarrow (MA) \quad \text{or,}$
 $(AL) \leftarrow (MA)$
 $(AH) \leftarrow (MA + 1)$

Or

Compare direct and indirect addressing modes in terms of flexibility and performance.

Direct Addressing Mode

Here, the **effective address** of the memory location is given directly in the instruction. The effective address is just a **16-bit number** written in the instruction itself.

Example:

MOV BX, [1354H]

MOV BL, [0400H]

The square brackets around 1354H denote the **contents of the memory location**. When executed, this instruction copies the data from memory location 1354H into BX.

In direct addressing, the instruction itself specifies the memory location (displacement/offset) of the operand. This means the CPU can directly locate the data inside the segment without extra calculation.

Register Indirect Addressing Mode

In this mode, the instruction specifies the **register name** which holds the **effective address (EA)**.

Registers used to hold EA are **BX, BP, SI, DI**.

The **DS register** is normally used for base address calculation (SS if BP is used).

Example:

MOV CX, [BX]

Operations:

EA = (BX)

BA = (DS) × 10H

MA = BA + EA

$(CX) \leftarrow (MA) \quad \text{or,}$

$(CL) \leftarrow (MA)$

$(CH) \leftarrow (MA + 1)$

Comparison: Direct vs Indirect Addressing

1. Direct Addressing

- Memory address (displacement) is written **directly in the instruction**.
- **Less flexible**, because the address is fixed in the instruction.
- **Faster** than indirect, since the CPU doesn't need to read another register for EA.

2. Indirect Addressing

- Instruction specifies a **register** that contains the memory address.
- **More flexible**, because the register content can change, allowing dynamic addressing (useful in arrays, loops).
- Slightly **slower** than direct, since it requires reading the register to get EA.

👉 In short:

- **Direct = faster, less flexible.**
- **Indirect = more flexible, slightly slower.**

5. What is the function of the PUSH and POP instructions?

5

Ans: The PUSH and POP instructions in the 8086 microprocessor are used to store and retrieve data from the stack, a special memory area used for temporary storage, subroutine calls, and interrupt handling. The stack operates on a Last-In-First-Out (LIFO) principle, meaning the last value pushed is the first one to be popped.

- **PUSH:** Copies a register or memory word (16-bit) onto the stack. Before storing, the Stack Pointer (SP) is decreased by 2 (for a word) to point to the new top of the stack, and then the value is stored at that address. This allows the processor to save temporary data or return addresses during subroutine calls.
- **POP:** Retrieves the top word from the stack into a register or memory location. After reading, the SP is increased by 2 to point to the next item. This restores previously saved data or return addresses, making function calls and interrupts manageable.

Or

Describe how the stack works in the 8086 microprocessor.

Ans: In the 8086 microprocessor, the stack is a special area of memory used for temporary storage of data, return addresses, and status information during program execution. It operates on a Last-In-First-Out (LIFO) principle, meaning the last item pushed onto the stack is the first one to be popped off. The Stack Segment (SS) register holds the base address of the stack, while the Stack Pointer (SP) keeps track of the top of the stack.

When a PUSH instruction is executed, the SP is decreased by 2 (for a 16-bit word), and the data is stored at the new top of the stack. Conversely, during a POP operation, data is read from the address pointed to by SP, and then SP is increased by 2. The stack is widely used for storing temporary data, passing parameters to subroutines, saving return addresses, and handling interrupts, which makes program execution organized and efficient.

Class Test – 3
CSE 3105 - Microprocessors and Embedded System

1. Write an assembly program to add two 16-bit numbers stored in memory locations NUM1 and NUM2, and store the result at location SUM. Assume the data segment is initialized. 4

```
.model small
.stack 100h
.data
    num1 dw 5
    num2 dw 3
    sum dw ?

.code
main proc
    mov ax, @data
    mov ds, ax

    mov ax, num1
    add ax, num2 ; for subtraction use "sub ax, num2"
    mov sum, ax

    ; convert result to ascii digit
    mov ax, sum
    add al, 48    ; make it ascii

    ; display it
    mov ah, 2
    mov dl, al
    int 21h

    ; exit
    mov ah, 4ch
    int 21h
main endp
end main
```

If we try Hex then

```
.model small
.stack 100h
.data
    num1 dw 1234h ; first 16-bit number
    num2 dw 4321h ; second 16-bit number
```

```

    sum dw ?      ; result will be stored here

.code
main proc
    mov ax, @data    ; initialize data segment
    mov ds, ax

    ; load num1 into ax
    mov ax, num1

    ; add num2 to ax
    add ax, num2

    ; store result into sum
    mov sum, ax

    ; exit program
    mov ah, 4ch
    int 21h
main endp
end main

```

Or

Write an assembly program to multiply two 8-bit numbers stored in NUM1 and NUM2 and store the 16-bit result in RESULT.

```

.model small
.stack 100h
.data
    NUM1 db 5      ; first 8-bit number
    NUM2 db 3      ; second 8-bit number
    RESULT dw ?    ; 16-bit result

.code
main proc
    mov ax, @data
    mov ds, ax      ; initialize data segment

    ; Load first number into AL
    mov al, NUM1
    ; Multiply with second number (NUM2)
    mov bl, NUM2
    mul bl           ; AL * BL → AX (16-bit result)

    ; Store result in RESULT
    mov RESULT, ax

```

```

; Exit program
mov ah, 4Ch
int 21h
main endp
end main

```

2. Explain the function of the following instructions: 4

- i) MOV AX, BX
- ii) MUL BL
- iii) SUB AL, 05H
- iv) JZ NEXT

Ans:

i) MOV AX, BX

- **Function:** Copies the contents of the BX register into the AX register.
- **Details:**
 - AX and BX are **16-bit general-purpose registers**.
 - After execution, whatever value was in BX is now also in AX.
 - **Example:** If BX = 1234H, after MOV AX, BX, AX = 1234H.
- **Note:** The original value in BX remains unchanged.

ii) MUL BL

- **Function:** Performs **unsigned multiplication** of the accumulator AL with the 8-bit register BL.
- **Details:**
 - AL is the lower 8 bits of AX.
 - The result is stored in AX (for 8-bit multiplication).
 - **Example:** If AL = 05H and BL = 03H, then after MUL BL, AX = 0FH ($5 \times 3 = 15$ decimal = 0FH).
- **Note:** MUL does **not affect the source operands**, but sets the flags if the result is larger than the destination.

iii) SUB AL, 05H

- **Function:** Subtracts the **immediate value** 05H from the AL register.
- **Details:**
 - If AL = 0AH before execution, then after SUB AL, 05H, AL = 05H.
 - Updates **flags** (Zero, Sign, Carry, Overflow) according to the result.

iv) JZ NEXT

- **Function: Jump if Zero** — transfers control to the label NEXT if the Zero Flag (ZF) is set.
- **Details:**
 - Usually used after arithmetic or comparison instructions.
 - **Example:** If the previous instruction made the result zero, JZ NEXT will jump to NEXT; otherwise, execution continues sequentially.

Or

Consider the following code snippet:

```
MOV AX, 0000H
MOV CX, 0005H
MOV BX, 0002H
LOOP1: ADD AX, BX
      INC BX
      LOOP LOOP1
```

Determine:

- i) The final value of AX in hexadecimal.
- ii) How many times the ADD instruction executes.

Ans:

Step 1: Understand instructions

1. MOV AX, 0000H → AX = 0
2. MOV CX, 0005H → CX = 5 (loop counter)
3. MOV BX, 0002H → BX = 2

LOOP instruction:

- LOOP LOOP1 → Decrements CX by 1, and if CX ≠ 0, jumps to LOOP1.

So the loop will execute **5 times**.

Step 2: Track values iteration by iteration

Iteration	CX before LOOP	AX before ADD	BX before ADD	ADD AX,BX result	BX after INC

1	5	0000	0002	0000 + 0002 = 0002	0003
2	4	0002	0003	0002 + 0003 = 0005	0004
3	3	0005	0004	0005 + 0004 = 0009	0005
4	2	0009	0005	0009 + 0005 = 000E	0006
5	1	000E	0006	000E + 0006 = 0014	0007

Step 3: Determine answers

i) Final value of AX (hex):

- AX = **0014H**

ii) Number of times ADD executes:

- The loop runs 5 times → **ADD executes 5 times**

3. Write an 8086 assembly program to read a character from the keyboard and display it back on the screen using DOS interrupt INT 21H.

```

.model small
.stack 100h
.code

main proc
    ; read a character from keyboard
    mov ah, 1    ; dos function 01h - read character
    int 21h     ; al = character entered

    ; display the same character back
    mov ah, 2    ; dos function 02h - display character
    mov dl, al   ; move input character into dl
    int 21h

    ; exit program
    mov ah, 4ch
    int 21h
main endp
end main

```

Or

Write an 8086 assembly program to display the message "HELLO" on the screen using DOS interrupt INT 21H.

```

.model small
.stack 100h
.data
msg db 'HELLO $'
.code

main proc
    mov ax, @data
    mov ds, ax

    mov ah, 9
    lea dx, msg
    int 21h

exit:
    mov ah, 4Ch
    int 21h
main endp
end main

```

4. Compare 8086 and 8088 microprocessors. 4

Ans:

Comparison between 8086 and 8088 Microprocessors

Feature	8086	8088
Execution Unit & Instruction Set	Same EU and instruction set	Same EU and instruction set
Bus Interface Unit (BIU)	Different from 8088	Different from 8086
Data Bus Width	16-bit (AD0–AD15, demultiplexed)	8-bit (AD0–AD7, demultiplexed)
Address Bus Width	20-bit	8-bit
Memory Organization	Two memory banks, each 512 KB	Single memory bank
Instruction Queue Size	6 bytes	4 bytes
Clock Speeds	5 / 8 / 10 MHz	5 / 8 MHz
Control Signals (Min. Mode)	Pin 28 = M/ $\overline{\text{IO}}$; BHE used for higher byte access	Pin 28 = IO/ $\overline{\text{M}}$; no BHE needed (8-bit bus only)

Or

Explain the basic features of Intel 8087 Co-processor.

Ans:

Basic Features of Intel 8087 Co-processor

- The **Intel 8087** is a **math co-processor** (also called **Numeric Data Processor, NDP**) specially designed to handle **complex mathematical calculations** involving integers and floating-point data.
- It works in **parallel with the 8086 microprocessor** in **maximum mode**.

Key Features:

1. Supports **integer, decimal, and real data types**, with operand sizes ranging from **2 bytes to 10 bytes**.
2. Provides a **rich instruction set**, including arithmetic operations (addition, subtraction, multiplication, division) and advanced functions like **square root, exponential, and trigonometric calculations**.
3. Offers **high performance**: for example, multiplication of two 64-bit real numbers takes about **27 μ s**, and calculating a square root takes about **36 μ s**.
4. Complies with the **IEEE floating-point standard**, ensuring accuracy and compatibility.
5. **Multi-bus compatible**, making it flexible to connect with system buses.